Tech Stack & Architecture Documentation

Project: Legal Document RAG Assistant

1. Architectural Overview

The application will utilize a decoupled architecture. A modern frontend (built to achieve the premium cinematic UI outlined in the Design Document) will communicate via REST API with a Python-based backend. The backend handles the PDF ingestion, vector database queries, and LLM orchestration, ensuring the strict "out-of-context" fallback rules are enforced.

2. Frontend Layer (UI/UX)

To achieve the "Cave-Moonlight" atmosphere, floating context cards, and smooth responsive states (like bottom-sheet modals on mobile), a robust frontend framework is required.

- **Framework: React.js** (or Next.js for easy routing and deployment).
  - *Why:* Allows for precise control over component state, responsive design (`100dvh` for mobile keyboards), and reusable chat bubble components.
- **Styling: Tailwind CSS**.
  - *Why:* Extremely efficient for implementing the exact hex codes (`#09100B` , `#265131` , `#51C445`) and glassmorphic (`backdrop-blur`) effects required by the design spec.
- **Animations: Framer Motion**.
  - *Why:* To handle the smooth entry of the floating "Context Citations" on PC and the drag-to-dismiss "Bottom Sheet" modals on mobile.
- **Icons: Lucide React**.
  - *Why:* Clean, lightweight, open-source icons that match the modern aesthetic.

3. Backend & Orchestration Layer

The backend is responsible for document parsing, reasoning, and enforcing the strict security and accuracy guardrails required for legal queries.

- **Language: Python 3.10+**
- **API Framework: FastAPI**.
  - *Why:* Lightning-fast, natively supports asynchronous endpoints (crucial for streaming LLM responses), and automatically generates Swagger documentation.
- **AI Orchestration: LangChain** (Python).

- *Why:* Provides built-in wrappers for `PyPDFLoader`, `RecursiveCharacterTextSplitter`, and Chroma integration. It makes implementing the *Pre-LLM Similarity Thresholding* (for the strict fallback) much easier.

4. Data & Vector Storage Layer

- **Vector Database: Chroma (ChromaDB)**.
  - *Why:* Meets the project requirement. It is open-source, runs locally (no cloud costs), and stores embeddings directly on the disk or in memory. It natively supports Cosine Distance calculations needed for the out-of-domain thresholding.
- **Document Parsing: PyPDF2** or **pdfplumber** (wrapped via LangChain's `PyPDFLoader`).

5. AI / Machine Learning Layer (100% Free)

As required, the system relies entirely on free, open-source models, avoiding paid APIs like OpenAI or Anthropic.

- **Embedding Model (Encoder):** `sentence-transformers/all-MiniLM-L6-v2` or `BAAI/bge-small-en-v1.5`.
  - *Source:* Hugging Face.
  - *Why:* These are highly optimized, lightweight models that convert the PDF chunks into high-quality vector embeddings. They run locally on the CPU instantly without any cost.
- **Generator Model (LLM/Regressive): Llama-3-8B-Instruct** (Meta) or **Mistral-7B**.
  - *Option A (Local execution):* Run via **Ollama**. (Best if your TCS workstation has 8GB+ RAM/VRAM. Completely offline and private).
  - *Option B (Cloud execution):* Run via **Groq API**. Groq provides a generous free tier for Llama 3 inference that runs at 800+ tokens per second. (Best if developing on a lower-spec laptop).

6. System Flow & Data Pipeline

1. **Ingestion:** The PDF (Public Law 111-5) is loaded into the Python backend.
2. **Chunking:** The text is split into overlapping chunks (~500 tokens).
3. **Embedding:** The Hugging Face encoder converts chunks into vectors.
4. **Storage:** Vectors and metadata (page numbers, exact text) are saved to Chroma.
5. **Query:** User enters a question in the React frontend.
6. **Retrieval & Threshold Check:**
   - Query is vectorized and searched in Chroma.

    - *Guardrail:* If the Cosine Distance of the top result is `> 0.4` (or chosen threshold), the backend immediately returns: *"Sorry I can't find anything relevant to this question."*
7. **Generation:** If the threshold is met, the Context and Query are injected into the strict system prompt and sent to the LLM (via Ollama or Groq).
8. **Delivery:** The LLM's response, along with the raw text of the Context Citation (for the floating UI cards), is returned to the React frontend.

7. Deployment (Optional for Internship)

If you need to host this to show your mentors:

- **Frontend:** Vercel (Free tier).
- **Backend:** Render or Railway (Free tier) to host the FastAPI server and Chroma instance.